Der Wetterstein – Programmierteil

IT4Teens · Dauer: ca. 2,5 Stunden · Für alle ohne Vorkenntnisse

---

Inhalt

Was baust du hier? 1
Was du brauchst 2
Die Verkabelung 2
Erste Schritte 3
Das musst du über Code wissen 3
Wenn etwas nicht klappt 4
Spickzettel 5
Und wenn ich früher fertig bin? 5
Und wenn ich hängenbleibe? 6
Aufgabe 1 – Das OLED-Display 7
Aufgabe 2 – Die RGB-LED 9
Aufgabe 3 – Der Taster 12
Aufgabe 4 – Der Wettersensor 15

Was baust du hier?

Am Ende des Tages hältst du eine **eigene kleine Wetterstation** in der Hand: Sie misst Temperatur, Luftfeuchtigkeit und Luftdruck, zeigt alles auf einem Display an und verrät dir mit einer farbigen LED auf einen Blick, wie das Wetter ist.

Das Ganze steuert ein **Mikrocontroller** – ein winziger Computer für rund 5 Euro, der keinen Bildschirm und keine Tastatur hat, dafür aber Sensoren, Lampen und Motoren direkt ansteuern kann. Solche Chips stecken in deiner Waschmaschine, in Ampeln, im Auto und in fast jedem Gerät, das irgendetwas „von selbst" macht.

Du gehst in **vier Schritten** vor. Jeder Schritt baut auf dem vorherigen auf:

#	Aufgabe	Was du lernst	Zeit
1	**OLED-Display**	Etwas ausgeben – der Computer redet mit dir	25 min
2	**RGB-LED**	Etwas ansteuern – der Computer bewegt die Welt	25 min
3	**Taster**	Etwas einlesen – die Welt redet mit dem Computer	30 min
4	**Wettersensor**	Alles zusammen – der fertige Wetterstein	40 min

Dazu kommen etwa 15 Minuten Einstieg und 15 Minuten Puffer.

> **Wo ist das WLAN?** Der Wetterstein kann seine Daten später auch ins Netz schicken. Dieser Teil ist ein eigener Baustein des Gesamtprojekts und hat sein eigenes Repository – er ist hier bewusst nicht dabei, sonst wird der Tag zu lang. → Link: *(hier das GitHub-Repo des WLAN-Bausteins eintragen)*

Was du brauchst

Hardware

- NodeMCU v2 (ESP8266) – das Board mit dem Mikrocontroller
- OLED-Display 128 × 64 (SSD1306)
- Wettersensor BME280
- RGB-LED (WS2812)
- Taster
- Micro-USB-Kabel

Software auf dem Rechner

- Visual Studio Code
- Erweiterung **PlatformIO IDE** (in VS Code links im Extensions-Menü suchen und installieren)

Die Verkabelung

Bauteil	Anschluss am Board
OLED-Display	SDA → **D2**, SCL → **D1**, VCC → 3V3, GND → GND
Wettersensor BME280	SDA → **D2**, SCL → **D1**, VCC → 3V3, GND → GND
RGB-LED (Data)	**D4**
Taster	**D3** und GND

Display und Sensor hängen an **denselben zwei Leitungen**. Das geht, weil sie sich den sogenannten **I2C-Bus** teilen: Jedes Bauteil hat eine eigene Adresse – eine Art Hausnummer –, und der Mikrocontroller spricht immer nur eines davon an. Der Sensor hört auf `0x76`, das Display auf `0x3C`.

> ℹ️ Diese Tabelle ergibt sich aus dem Code. Wenn euer Bausatz anders verkabelt ist, gilt natürlich euer Aufbau – dann müssen im Code die Pin-Nummern angepasst werden.

Erste Schritte

1. Das richtige Projekt öffnen

> ⚠️ **Das ist die wichtigste Regel des Tages.** Öffne in VS Code immer **genau einen Aufgabenordner**, zum Beispiel `Aufgabe 1 - OLED` – **nicht** den Ordner darüber. PlatformIO erkennt ein Projekt nur an der Datei `platformio.ini`, und die liegt in jedem Aufgabenordner einzeln.

Datei → Ordner öffnen… → `Aufgabe 1 - OLED` → Öffnen

Beim allerersten Mal lädt PlatformIO im Hintergrund einige Pakete herunter. Das kann ein paar Minuten dauern – unten rechts läuft ein Fortschrittsbalken. Einfach warten.

2. Die drei Knöpfe, die du brauchst

Ganz unten in VS Code läuft eine blaue Leiste. Dort sind sie:

Symbol	Name	Was es tut	Tastenkürzel
✓ Häkchen	**Build**	Übersetzt deinen Code. Findet Tippfehler – **ohne** ihn aufs Board zu laden.	`Strg` + `Alt` + `B`
→ Pfeil	**Upload**	Übersetzt **und** überträgt aufs Board.	`Strg` + `Alt` + `U`
🔌 Stecker	**Serial Monitor**	Zeigt Texte, die das Board zurückschickt.	–

Faustregel: Nach jeder kleinen Änderung einmal **Build** drücken. So findest du einen Fehler sofort und weißt genau, welche Zeile schuld ist. Wenn du eine Stunde lang schreibst und dann erst baust, hast du zwanzig Fehler auf einmal.

3. Loslegen

Öffne im Ordner `src` die Datei mit dem Programm und suche nach **`TODO`**. Nur dort musst du etwas schreiben. Alles andere ist schon fertig.

Zu jeder Aufgabe gibt es außerdem ein Aufgabenblatt: **`AUFGABE.md`** im jeweiligen Ordner. Dort steht ausführlich, worum es geht.

Das musst du über Code wissen

Jedes Programm hat zwei Teile

```
void setup() {
  // Läuft GENAU EINMAL, direkt nach dem Einschalten.
  // Hier wird alles vorbereitet.
}

void loop() {
  // Läuft danach IMMER WIEDER, endlos schnell,
  // bis der Strom weg ist. Hier passiert die Arbeit.
}
```

Ein Mikrocontroller ist nie „fertig". Er wiederholt `loop()` viele tausend Mal pro Sekunde – und genau daraus entsteht alles, was das Gerät tut.

Die vier Regeln, an denen alle zuerst scheitern

1. **Jede Anweisung endet mit einem Semikolon `;`** Vergisst du es, meckert der Compiler meist in der **nächsten** Zeile. Schau also auch immer eine Zeile darüber.
2. **Jede geöffnete Klammer `{` braucht eine schließende `}`** VS Code hilft: Klick auf eine Klammer, dann leuchtet die Partnerklammer mit auf.
3. **Groß- und Kleinschreibung zählt.** `FastLED` ist etwas anderes als `fastled`. Der Computer ist da gnadenlos.
4. **Text in Anführungszeichen, Zahlen ohne.** `oled.print("Hallo");` aber `oled.print(23);`

Kommentare

```
// Alles hinter zwei Schrägstrichen ist eine Notiz für Menschen.
/* Das geht auch
   über mehrere Zeilen. */
```

Der Computer überliest das komplett. Kommentare sind für dich und für die, die deinen Code später lesen – auch für dich selbst in drei Wochen.

Wenn etwas nicht klappt

Fehler beim Übersetzen (rote Meldungen)

Lies die **erste** rote Zeile. Dort stehen Dateiname, Zeilennummer und Problem. Alles darunter sind meistens nur Folgefehler. Häufig:

Meldung	Bedeutet
`expected ';' before ...`	Semikolon fehlt – meist eine Zeile **darüber**
`'xyz' was not declared in this scope`	Der Name ist unbekannt: vertippt, oder die Variable wurde noch nicht angelegt
`expected '}' at end of input`	Eine schließende Klammer fehlt

Upload schlägt fehl

- Steckt das USB-Kabel richtig? Manche Kabel können nur laden, nicht Daten übertragen – probiere ein anderes.
- Ist der Serial Monitor noch offen? Der belegt den Anschluss. Erst schließen, dann hochladen.
- Board kurz ab- und wieder anstecken, dann erneut probieren.

Das Programm lädt, aber es passiert nichts

Das ist selten ein Defekt. Fast immer fehlt einer der beiden „Abschicken"-Befehle:

- Display: `oled.update()`
- LED: `FastLED.show()`

Ohne die bleibt alles nur im Speicher – und damit unsichtbar.

Spickzettel

```
// --- Display ---
oled.clear();                  // Bild löschen
oled.setCursor(spalte, zeile);// Position: spalte 0-127, zeile 0-7
oled.print("Text");            // schreiben
oled.print(zahl);              // Zahl schreiben
oled.setScale(2);              // Schriftgröße 1 bis 4
oled.update();                 // ANZEIGEN – nicht vergessen!

// --- RGB-LED ---
wetterLeds[0] = CRGB::Red;     // Farbe wählen
wetterLeds[0] = CRGB(r, g, b);// eigene Mischung, je 0-255
FastLED.setBrightness(80);     // Helligkeit 0-255
FastLED.show();                // ANZEIGEN – nicht vergessen!

// --- Taster ---
digitalRead(BUTTON_PIN)        // LOW = gedrückt

// --- Sensor ---
bme.readTemperature();         // Grad Celsius
bme.readHumidity();            // Prozent
bme.readPressure() / 100.0;    // Hektopascal

// --- Allgemein ---
delay(500);                    // 500 ms warten
millis();                      // Millisekunden seit dem Start
Serial.println("Hallo");       // Text an den Computer
```

Und wenn ich früher fertig bin?

Kein Problem – **jede Aufgabe hat Extras**. Sie stehen unten in der Code-Datei und im Aufgabenblatt und sind mit ⭐ markiert. Such dir aus, was dich interessiert; die Reihenfolge ist egal, und niemand muss alle machen.

Die Extras sind keine Beschäftigungstherapie: In Aufgabe 2 lernst du, wie man zwanzig fast gleiche Zeilen zu vier zusammenschrumpft. In Aufgabe 4 baust du einen stufenlosen Farbverlauf. Das ist echtes Handwerk.

Und wenn ich hängenbleibe?

1. Lies die Fehlermeldung – ernsthaft, die erste Zeile.
2. Schau ins Aufgabenblatt `AUFGABE.md`, ganz unten steht eine Fehlertabelle.
3. Frag die Person neben dir. Fremden Code zu lesen ist die beste Übung.
4. Frag die Betreuerin oder den Betreuer.
5. Im Ordner `loesung` liegt die Musterlösung. Sie ist kein Betrug – schau hinein, **verstehe** die Zeile, die dir fehlte, und tippe sie selbst ab.

> ⚠️ Kopiere nicht die ganze Lösungsdatei in den `src`-Ordner. Dann gibt es `setup()` und `loop()` doppelt und nichts geht mehr. Übertrage nur die Zeilen, die dir fehlen.

Aufgabe 1 – Das OLED-Display

Zeit: ca. 25 Minuten · **Datei zum Bearbeiten:** `src/main_OLED.cpp`

Worum geht es?

Der Wetterstein soll später Temperatur und Luftfeuchtigkeit anzeigen. Dafür brauchen wir zuerst ein Display – und du bringst es zum Leben.

Das kleine schwarze Display ist **128 Pixel breit** und **64 Pixel hoch**. Es hängt an nur zwei Datenleitungen (**I2C**: SDA und SCL) am Mikrocontroller. Über diese zwei Drähte schicken wir alles, was angezeigt werden soll.

Was du wissen musst

Ein Programm für den Mikrocontroller hat immer zwei Teile:

Teil	Wann läuft er?	Wofür?
`setup()`	genau **einmal** beim Einschalten	Sachen vorbereiten, z. B. Display anschalten
`loop()`	**immer wieder**, endlos	die eigentliche Arbeit

Der Punkt-Operator: `oled` ist unser Display. Mit `oled.clear()` sagen wir „Display, lösche dich!". Der Punkt bedeutet also: *Dieser Befehl gilt für dieses Ding.*

Das Display ist wie eine Schreibmaschine:

1. `oled.clear()` – Blatt leeren
2. `oled.setCursor(spalte, zeile)` – hinstellen *spalte* = 0…127 (Pixel von links), *zeile* = 0…7 (Textzeile von oben)
3. `oled.print("Text")` – schreiben
4. `oled.update()` – **abschicken**

> ⚠️ Ohne `oled.update()` bleibt das Display leer! Das ist der häufigste Fehler.

Deine Aufgaben

TODO 1.1 – Display startklar machen (in `setup()`)

Schalte das Display ein und lösche seinen Inhalt.

TODO 1.2 – Text anzeigen (in `loop()`)

Zeige einen eigenen Text an, zum Beispiel deinen Namen. In eine Zeile passen ungefähr **21 Zeichen**.

So sieht es aus, wenn es klappt

Auf dem Display steht dein Text – dauerhaft und ohne Flackern.

Extras für Schnelle

Die Extras stehen unten in der Code-Datei. Reihenfolge egal, such dir aus, was dich reizt:

- **A – Große Schrift:** `oled.setScale(2);` vor dem `print`. Was macht `setScale(4)`?
- **B – Zähler:** eine Variable, die bei jedem Durchlauf um 1 hochzählt.
- **C – Bild malen:** `rect()`, `line()`, `circle()`, `dot()` – zeichne ein Smiley oder ein Haus.
- **D – Laufschrift:** lass den Text von rechts nach links durchs Display wandern.

Wenn etwas nicht klappt

Problem	Wahrscheinliche Ursache
Display bleibt komplett dunkel	`oled.init()` fehlt, oder Kabel/Stecker sitzt nicht
Display leuchtet, aber kein Text	`oled.update()` vergessen
Text ist abgeschnitten	zu lang – kürzer schreiben oder auf zwei Zeilen verteilen
Fehler beim Kompilieren	fehlendes Semikolon `;` oder eine Klammer zu wenig – schau auf die **erste** rote Zeile

Aufgabe 2 – Die RGB-LED

Zeit: ca. 25 Minuten · **Datei zum Bearbeiten:** `src/main_LED.cpp`

Worum geht es?

Der Wetterstein soll schon von weitem zeigen, wie das Wetter ist – ohne dass man das Display lesen muss. Dafür sorgt eine **RGB-LED**.

In dieser einen LED stecken in Wirklichkeit **drei winzige Lämpchen**: ein rotes, ein grünes und ein blaues. Jedes davon kann in 256 Helligkeitsstufen leuchten (0 = aus, 255 = volle Kraft). Zusammen ergibt das über **16 Millionen Farben** – genau wie bei deinem Handybildschirm.

Rot	Grün	Blau	Ergebnis
255	0	0	Rot
255	255	0	Gelb
0	255	255	Türkis
255	255	255	Weiß
0	0	0	aus

Was du wissen musst

`#define` gibt einer Zahl einen Namen:

```
#define LED_PIN 2
```

Überall, wo später `LED_PIN` steht, setzt der Computer beim Übersetzen die `2` ein. Vorteil: Steckst du die LED an einen anderen Pin, änderst du nur **eine einzige Zeile**.

Die eckigen Klammern `[ ]` bedeuten „Liste":

```
CRGB wetterLeds[NUM_LEDS];
```

Das ist ein Platz für die Farbe jeder LED. Die **erste** LED heißt `wetterLeds[0]` – ja, mit Null. Programmierer fangen beim Zählen fast immer bei 0 an. Daran gewöhnst du dich schneller, als du denkst.

Farbe setzen geht immer in drei Schritten:

```
wetterLeds[0] = CRGB::Red;   // 1. Farbe aussuchen
FastLED.show();              // 2. an die LED schicken
delay(500);                  // 3. halbe Sekunde warten
```

> ⚠️ Ohne `FastLED.show()` passiert **nichts**. Das ist hier der Klassiker unter den Fehlern.

Das `::` heißt **Scope-Operator** und bedeutet: „nimm `Red` aus der Farbsammlung `CRGB`".

Deine Aufgaben

TODO 2.1 – Die LED anmelden

Schreibe die drei Zeilen, die Pin, Anzahl und den Farbspeicher festlegen.

> ⚠️ **Wichtig:** Solange TODO 2.1 offen ist, lässt sich das Programm **nicht** übersetzen. Du bekommst dann diese Meldung:
>
> ```
> 'wetterLeds' was not declared in this scope
> ```
>
> Das ist kein Defekt, sondern genau richtig. Übersetzt heißt es: *„Du benutzt weiter unten etwas namens `wetterLeds`, aber du hast mir nie gesagt, was das sein soll."* **Merk dir das Vorgehen:** Bei einer Fehlermeldung immer die **erste** rote Zeile lesen. Dort stehen Datei, Zeilennummer und das eigentliche Problem. Alles darunter sind meist nur Folgefehler.

TODO 2.2 – Farben nacheinander anzeigen

Baue eine Abfolge aus **mindestens 5 Farben** – zum Beispiel einen Regenbogen.

Farben zur Auswahl: `Red` `Orange` `Gold` `Yellow` `Green` `SpringGreen` `Aqua` `Cyan` `Blue` `RoyalBlue` `Purple` `Magenta` `Pink` `White` `Amethyst` `Black`

> 💡 Tipp: Tippe `CRGB::` und warte kurz – VS Code schlägt dir alle über 100 Farben vor.

So sieht es aus, wenn es klappt

Die LED wechselt im halben Sekundentakt durch deine Farben und fängt dann wieder von vorne an.

Extras für Schnelle

- **A – Kürzer statt länger:** Ersetze die vielen fast gleichen Zeilen durch eine **Farbliste + `for`-Schleife**. Gleiches Ergebnis, ein Drittel Code. *(Das ist das wichtigste Extra – hier lernst du am meisten.)*
- **B – Weicher Regenbogen:** `CHSV(farbton, 255, 255)` dreht den Farbton stufenlos weiter, statt zwischen festen Farben zu springen.
- **C – Atmen:** LED langsam heller und wieder dunkler werden lassen.
- **D – Zufall:** `random(0, 256)` für jede Grundfarbe – jedes Mal eine neue Farbe.

Wenn etwas nicht klappt

Problem	Wahrscheinliche Ursache

Problem	Wahrscheinliche Ursache
LED bleibt dunkel	`FastLED.show()` vergessen, oder `LED_PIN` stimmt nicht
LED zeigt nur eine Farbe	`delay()` fehlt – die Farben wechseln zu schnell zum Sehen
LED blendet extrem	`FastLED.setBrightness()` runtersetzen, z. B. auf 40
Farbe stimmt nicht ganz	manche LEDs vertauschen Rot und Grün – nicht schlimm, probier eine andere Farbe
`'wetterLeds' was not declared`	TODO 2.1 fehlt oder steht **unter** `setup()` statt darüber

Aufgabe 3 – Der Taster

Zeit: ca. 30 Minuten · **Datei zum Bearbeiten:** `src/main_Button.cpp`

Worum geht es?

Bis jetzt hat der Wetterstein einfach sein Programm heruntergespult. Jetzt soll er auf **dich** reagieren: Bei jedem Druck auf den Taster wechselt die LED die Farbe.

Damit lernst du zwei Dinge, die du in fast jedem Programm wiedersehen wirst: **Eingaben lesen** und **zwischen Zuständen umschalten**.

Was du wissen musst

Warum ein Taster „prellt"

Im Taster sind zwei Metallteile. Beim Drücken springen die für ein paar Millisekunden wie ein Flummi hin und her – also zigmal an, aus, an, aus.

Dein Auge merkt davon nichts. Der Mikrocontroller schon: Er schaut **tausende Male pro Sekunde** nach und würde jedes Zappeln als eigenen Druck zählen. Die LED würde bei einem Druck wild durch alle Farben rasen.

Die Lösung heißt **Entprellen** (englisch *debouncing*): Nach der ersten Änderung ignorieren wir 50 Millisekunden lang alles Weitere. Dann ist das Zappeln vorbei – und ein Druck ist wirklich ein Druck.

> 👍 Gute Nachricht: Das Entprellen ist schon fertig für dich programmiert, in der Funktion `tasterWurdeGedrueckt()`. Lies sie dir ruhig durch, aber du musst sie nicht anfassen.

`INPUT_PULLUP` – warum ist gedrückt gleich `LOW`?

Der Pin ist von sich aus auf `HIGH` (Strom liegt an). Der Taster verbindet ihn beim Drücken mit Masse, also `LOW`. Klingt verdreht, ist aber bei Tastern der Normalfall – so spart man sich einen extra Widerstand auf der Platine.

Gedrückt = `LOW`.

Der `switch` – eine Weiche

```
switch (farbNummer) {
  case 0:
    wetterLeds[0] = CRGB::Red;
    break;
  case 1:
    wetterLeds[0] = CRGB::Orange;
    break;
}
```

Je nachdem, welchen Wert `farbNummer` gerade hat, wird **genau ein** Fall ausgeführt.

> ⚠️ Das `break;` am Ende jedes Falls ist Pflicht. Ohne `break` läuft das Programm einfach in den nächsten Fall weiter – ein Fehler, den auch Profis regelmäßig machen.

Deine Aufgaben

TODO 3.1 – Anzahl der Farben festlegen

Trage bei `ANZAHL_FARBEN` ein, wie viele Fälle dein `switch` haben wird.

> ⚠️ Wenn du später eine Farbe dazu tust, musst du diese Zahl **mit erhöhen** – sonst wird deine neue Farbe nie erreicht.

TODO 3.2 – Den `switch` vervollständigen

Zwei Fälle sind als Beispiel schon da. Ergänze die restlichen.

So sieht es aus, wenn es klappt

Jeder Druck schaltet **eine** Farbe weiter. Nach der letzten Farbe geht es wieder bei der ersten los. Im Serial Monitor siehst du bei jedem Druck eine Zeile.

> 💡 **Serial Monitor öffnen:** unten in VS Code auf das Stecker-Symbol (*Serial Monitor*) in der blauen Leiste klicken.

Extras für Schnelle

- **A – Aus-Zustand:** ein Fall, in dem die LED aus ist (`CRGB::Black`).
- **B – Zähler:** mitzählen, wie oft insgesamt gedrückt wurde, und ausgeben.
- **C – Helligkeit statt Farbe:** Stufen 0 / 20 / 60 / 150 / 255 durchschalten.
- **D – Langer Druck:** über 1 Sekunde gehalten → zurück auf Farbe 0.
- **E – Zwei Bausteine zusammen:** den Farbnamen zusätzlich aufs Display schreiben. *(Das ist schon fast der fertige Wetterstein.)*

Wenn etwas nicht klappt

Problem	Wahrscheinliche Ursache
LED springt bei einem Druck mehrere Farben weiter	`tasterWurdeGedrueckt()` verändert – Original wiederherstellen

Problem	Wahrscheinliche Ursache
Eine Farbe wird übersprungen / LED bleibt dunkel	`ANZAHL_FARBEN` passt nicht zur Anzahl der `case`-Fälle
Alle Farben laufen von allein durch	ein `break;` fehlt
Taster reagiert gar nicht	Kabel prüfen; ist es wirklich Pin **D3**?
Upload schlägt fehl	Taster beim Einstecken/Reset **nicht** gedrückt halten – D3 ist beim Start ein Sonderpin

Aufgabe 4 – Der Wettersensor

Zeit: ca. 40 Minuten · **Datei zum Bearbeiten:** `src/main_BME.cpp`

Worum geht es?

Das ist die Königsaufgabe: Hier kommt **alles zusammen**, was du bisher gelernt hast. Der Sensor misst das Wetter, das Display zeigt die Werte, und die LED verrät auf einen Blick, wie feucht die Luft ist. Danach ist dein Wetterstein fertig.

Was du wissen musst

Der Sensor: BME280

Ein Chip, kleiner als dein Fingernagel, der drei Dinge gleichzeitig misst:

Messgröße	Einheit	typischer Wert im Zimmer
Temperatur	°C	20 – 25
Luftfeuchtigkeit	%	35 – 60
Luftdruck	hPa	980 – 1030

Die **Höhe** misst er nicht wirklich – er *rechnet* sie aus dem Luftdruck aus. Warum das nur ungefähr stimmt, steht in Extra E.

`float` – Zahlen mit Komma

```
float temperatur = 23.4;
```

`int` kann nur ganze Zahlen (23). `float` kann auch 23.4 – und genau das braucht man bei Messwerten. Im Code schreibt man den Dezimal**punkt**, nicht das Komma.

Pascal und Hektopascal

Der Sensor liefert den Druck in **Pascal** (etwa 101300). Im Wetterbericht steht aber **Hektopascal** (etwa 1013). `1 hPa = 100 Pa` – deshalb `/ 100.0`.

Die `if`-Kette

```
if (luftfeuchte < 30) {
  wetterLeds[0] = CRGB::Red;
}
else if (luftfeuchte < 50) {
  wetterLeds[0] = CRGB::Gold;
}
else {
  wetterLeds[0] = CRGB::Blue;
}
```

Der Computer prüft **von oben nach unten** und nimmt den **ersten** Fall, der zutrifft. Deshalb genügt immer nur die obere Grenze – „und größer als 30" musst du nicht dazuschreiben, das ergibt sich von selbst.

> 💡 Das letzte `else` **ohne** Bedingung fängt alles ab, was übrig bleibt. So bleibt die LED nie dunkel, egal was der Sensor misst. In der alten Version dieses Programms fehlte genau das – bei 47 % Luftfeuchtigkeit ging die LED einfach aus, und niemand wusste warum.

Deine Aufgaben

TODO 4.1 – Den Sensor auslesen

Vier Variablen stehen schon da, überall mit einer `0`. Ersetze jede `0` durch den passenden Sensor-Befehl.

> 💡 Lade das Programm ruhig **jetzt schon** hoch und öffne den Serial Monitor. Solange dort überall `0.00` steht, weißt du: Diese Aufgabe ist noch offen. Sobald echte Zahlen erscheinen, hast du sie geschafft.

TODO 4.2 – Die Werte auf das Display bringen

Vier Zeilen (0, 2, 4 und 6), jeweils mit Bezeichnung und Einheit.

TODO 4.3 – Die LED zeigt die Luftfeuchtigkeit

Eine `if`/`else if`/`else`-Kette mit **mindestens 4 Stufen**.

So sieht es aus, wenn es klappt

Auf dem Display stehen vier aktuelle Messwerte, die sich jede Sekunde aktualisieren. Die LED leuchtet passend zur Luftfeuchtigkeit. **Der Wetterstein lebt.**

Extras für Schnelle

- **A – Hauch mal drauf:** Atme aus etwas Abstand Richtung Sensor. Wie schnell reagiert die Feuchte, wie lange dauert der Weg zurück? *(Sensor dabei nicht anfassen.)*
- **B – Stufenlose Farbe:** `map()` + `CHSV()` statt fester Stufen – die Farbe wandert fließend von Rot über Grün nach Blau. *(Das eindrucksvollste Extra.)*
- **C – Höchst- und Tiefstwert:** merken und anzeigen.
- **D – Zwei Display-Seiten mit dem Taster:** Aufgabe 3 und 4 kombiniert.
- **E – Denk mal nach:** Warum stimmt die angezeigte Höhe nie? Antwort steht in der Code-Datei ganz unten.

Wenn etwas nicht klappt

Problem	Wahrscheinliche Ursache
Display zeigt „BME nicht gefunden"	Sensor-Stecker sitzt nicht, oder er hat die Adresse `0x77` statt `0x76` – im Code ändern und testen
Alle Werte sind `0.00`	TODO 4.1 ist noch offen
Druck ist etwa `101300` statt `1013`	`/ 100.0` fehlt
Zahlen überlagern sich / Reste alter Zahlen	`oled.clear()` am Anfang von TODO 4.2 fehlt
Display bleibt leer	`oled.update()` am Ende vergessen
Temperatur ist 2–3 °C zu hoch	normal! Der Chip sitzt neben der warmen Platine und heizt sich selbst mit
LED geht bei manchen Werten aus	das abschließende `else` fehlt